This document outlines the telemetry data schema designed to identify throughput bottlenecks across the SyncOS mixed fleet of 5 autonomous forklifts and 5 dual-conveyor robots. The schema purposefully avoids generic metric dumps, focusing strictly on data points required to separate actionable software inefficiencies (orchestration) from physical site limitations (hardware, integrations, or operating processes).

The tables below map directly to the task execution lifecycle and fleet state, defining why each data point is captured and the specific product decision it informs.

Table A: Task Execution Log

This table tracks the end-to-end lifecycle of every material movement request. It is designed to expose delays in the current first-in, first-out (FIFO) assignment logic and measure the true operational cycle time versus idle wait time.

COLUMN NAME	RATIONALE (WHY IT IS NEEDED)	DECISION IT INFORMS
`Task_ID`	Uniquely identifies the material movement request generated by customer systems or PLCs.	Acts as the primary join key for relational analysis against fleet state data.
`Task_Type`	Differentiates between pallet handling (forklift) and material moves (conveyor).	Defines fleet eligibility parameters to evaluate if the correct vehicle class is being underutilized.
`Time_Created`	Captures the exact timestamp when the downstream system (WMS/ERP) requests the move.	Measures integration latency and total task queue depth to identify system-level bottlenecks.
`Time_Assigned`	Captures the timestamp when SyncOS allocates the task to a specific robot.	Highlights systemic delays in the current chronological FIFO assignment logic.
`Robot_ID`	Identifies the specific vehicle executing the assigned task.	Informs decisions on individual robot utilization rates and hardware-specific performance degradation.
`Time_Robot_Arrived_Pick`	Captures the timestamp when the robot physically arrives at the designated pickup station.	Calculates "Empty Travel Time" (deadheading) to determine if tasks are being assigned to vehicles geographically too far away.
`Time_Pick_Complete`	Captures the timestamp when the robot successfully loads the payload from the station.	Calculates "Station Wait/Dwell Time," determining if the physical station or human operators are

Table B: Fleet State & Traffic Log

This table captures periodic snapshots of the entire fleet. It provides the spatial and state-based context necessary to understand what robots are doing when they are not actively executing a loaded task.

COLUMN NAME	RATIONALE (WHY IT IS NEEDED)	DECISION IT INFORMS
`Timestamp`	Provides a high-frequency periodic snapshot (e.g., every 10 seconds) of robot telemetry.	Enables time-series analysis to calculate total fleet availability across the three production shifts.
`Robot_ID`	Links the categorical state and spatial data back to a specific vehicle.	Correlates individual robot behavior patterns with the broader task execution timeline.
`Current_State`	Categorizes the robot's real-time status (e.g., Moving_Empty, Moving_Loaded, Blocked_Traffic, Charging, Faulted).	Isolates the largest sources of non-productive time, differentiating between traffic deadlocks and empty travel routing issues.
`Location_Zone`	Captures physical coordinates (X, Y) or logical facility zones (e.g., Staging, Rack Storage).	Identifies spatial bottlenecks, high-congestion intersections, and sub-optimal staging layouts.
`Battery_Level`	Tracks the real-time state of charge for the vehicle.	Determines if excessive shift time is lost to unoptimized charging schedules rather than operational constraints.

> **Methodology Note: The Counter-Pattern**
>
> During analysis, this dataset is structured to deliberately check for false positives. By cross-referencing `Time_Robot_Arrived_Pick` and `Time_Pick_Complete` against shift schedules, we can isolate patterns where robots wait idly due to batch-releasing by human operators. This prevents misattributing throughput losses to software routing when the root cause is physical site processes.